

## أسئلة الجزء الأول

ليه لما بتعرف Constructor Custom ده بيلغي الـ Constructor Default في C#؟  
لما بتعمل Constructor Custom، الكومبايلر بيفهم إنك مش عايز أي Object يتكرت إل بشروطك وبالمعطيات اللي إنت محددها. عشان كده بيلغي الـ Constructor Default (التلقائي) اللي من غير Parameters (عشان يمنع إن الـ Object يتعمله تهيئة) Initialization) بشكل ناقص أو غير متوافق مع اللوجيك بتاعك.

إزاي الـ Overloading Method بيحسن قراءة الكود وإعادة استخدامه؟  
بيخليك تقدر تستخدم نفس اسم الدالة ( زي Sum ) عشان تعمل عمليات منطقية متشابهة لكن بأنواع أو عدد بيانات مختلف. ده بيخلي الكود أنصف وأسهل في القراءة بدل ما تضطر تخترع اسم جديد لكل دالة، وبكده بتوحد المسميات للوظائف المتشابهة.

إيه الهدف من الـ Chaining Constructor في الوراثة؟  
هدفه ضمان إن الكلاس ألب يتعمله تهيئة صحيحة للخصائص بتاعته قبل ما الكلاس البن يبدأ يهيئ أصالا في الخصائص الإضافية الخاصة بيه. ده بيمنع تكرار الكود ألنك بتعيد استخدام كود التهيئة اللي مكتوب ألب.

إيه الفرق بين new و override في الـ Overriding Method؟  
كلمة override بتعدل الدالة الأصلية الموجودة في ألب . ولما تستدعيها، النسخة الخاصة بالبن هي اللي هتتفد دائما، حتى لو كنت بتتعامل مع الـ Object عن طريق Reference من نوع ألب.  
أما كلمة new بتخفي دالة ألب وتعمل دالة جديدة

تماما ومستقلة. لو استدعيته من خالل Reference لألب، ا هينفذ نسخة ألب، ولو من خالل Reference لالبن، هينفذ نسخة البن.  
دايما بنعمل Override للـ ToString (في الكالسات بتاعتنا؟) ليه  
ألنها بشكلها الافتراضي بترجع اسم الكلاس فقط. لما بنعملها Override، بنقدر نخليها ترجع نص مفيد ومقروء يعبر عن حالة الكائن الحالية والبيانات اللي جواه.

ليه مينفعش تاخد Instance (كائن) من Interface بشكل مباشر؟  
ألن الـ Interface عبارة عن عقد أو تصميم بيحتوي على أسماء الدوال بس من غير أي كود فعلي لتنفيذها. فلو تم إنشاء كائن منه، مش هيكون عنده أي منطق لتنفيذ الدوال دي.

إيه فوايد الـ Implementations Default في الـ Interfaces اللي ظهرت في C# 8.0؟  
الفائدة الرئيسية هي التوافقية. بتسمحلك تضيف دالة جديدة للـ Interface وتحطيلها كود افتراضي من غير ما

تعطل الكالسات القديمة اللي بتعمل Implement لل Interface ده، ألنها هتستخدم الكود الافتراضي لو مكتبتش كود خاص بيها.

ليه من المفيد استخدام Reference Interface عشان نوصل لدوال الكالس اللي بينفذه؟  
ده بيحقق مبدأ الارتباط الضعيف. يعني بتكتب كود بيعتمد على العقد نفسه مش على كالس محدد. ده بيسمحلك تبديل الكالسات ببعضها بسهولة طالما إنها بتنفذ نفس ال Interface.

إزاي #C بتتغلب على قيود الوراثة المفردة باستخدام ال Interfaces؟  
لغة #C بتمنع إن كالس يورث من أكثر من كالس واحد عشان تتجنب مشاكل التضارب، لكنها بتسمح للكالس إنه يعمل Implement لعدد ل نهائي من ال Interfaces. ده بيدي للكالس القدرة على اكتساب خصائص متعددة.

إيه الفرق بين ال Method Virtual وال Method Abstract؟  
ال virtual بيكون ليها كود أساسي مبدئي في الكالس ألب، والكالس البن مخير إنه يسبب الكود زي ما هو أو يعمل له Override.  
أما ال abstract عبارة عن اسم وتوقيع بس من غير أي كود في الكالس ألب، والكالس البن مجبر إنه يكتب كودها ويعملها Override.

أسئلة الجزء الثاني  
إيه الفرق بين ال class وال struct في #C؟  
ال class يعتبر Type Reference (بيتخزن في ال Heap)، بيدعم الوراثة، ولما بتساويه بمتغير ثاني بياخد مرجع لنفس المكان.

ال struct يعتبر Type Value (بيتخزن في ال Stack)، ل يدعم الوراثة (إل ال Interfaces)، ولما بتساويه بمتغير ثاني بياخد نسخة مستقلة من البيانات.  
لو الوراثة عالقة بين الكالسات، وضح العالقات التانية الممكنة:  
بجانب الوراثة (A-Is)، فيه عالقات تانية أهمها:  
عالقة Association وهي عالقة استخدام عامة بين كالسين.  
عالقة A-Has (Aggregation) وهي عالقة احتواء ضعيفة؛ كالس بيحتوي على كالس ثاني بس الجزء يقدر يعيش لوحده.

عالقة Composition وهي عالقة احتواء قوية؛ كالس بيحتوي على كالس ثاني ول يمكن للجزء إنه يعيش بدون الكل.

عالقة Dependency وهي عالقة اعتماد مؤقتة.

### أسئلة الجزء الثالث

إيه هو الـ Binding Static والـ Binding Dynamic؟

الـ Binding Static بيحصل وقت الـ time-Compile. الكومبايلر بيكون عارف

مسبقا وبدقة أنهى دالة ا

هتتنفذ. من أشهر أمثله الـ Overloading Method.

بناء على النوع أما الـ Binding Dynamic بيحصل وقت الـ time-Run. تحديد الدالة اللي هتتنفذ بيتم

الفعلي للكائن. من أشهر أمثله الـ Overriding Method.